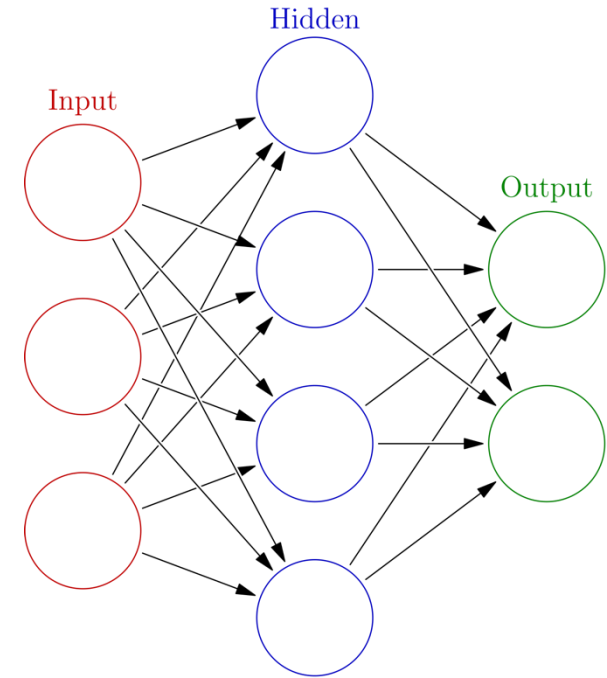


Classification with Neural Networks

MLP & CNNs

Week 5 Laboratory · 2 hours

Learning with Neural Networks



1

Understand NNs

2

Classify with MLP

3

Classify with CNNs

Learning Objectives

By the end of this lab, you will be able to:

1 **Explain** how an MLP performs classification

2 **Describe** the role of linear layers, ReLU & Softmax

3 **Explain** how networks learn via backprop & gradient descent

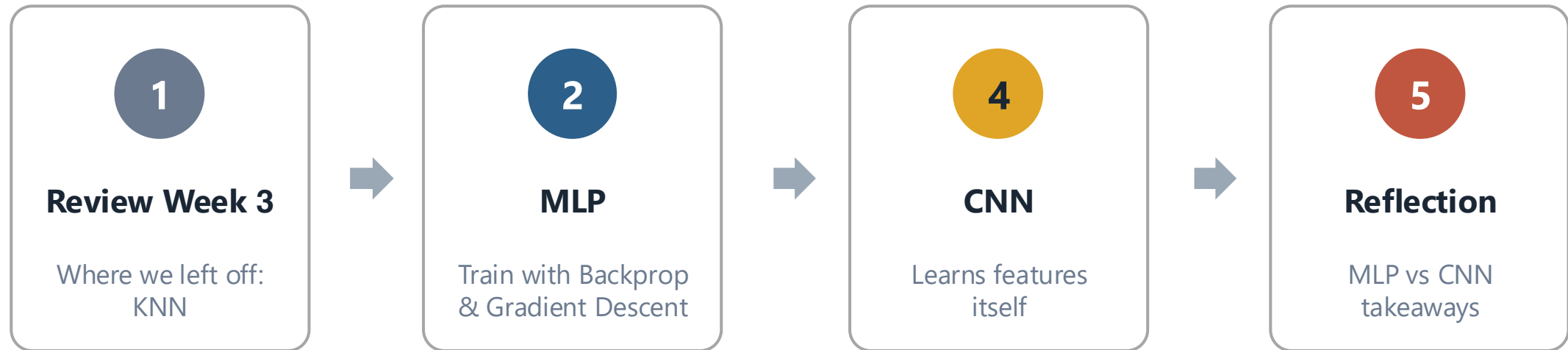
4 **Build** an MLP classifier using Keras

5 **Explain** why CNNs outperform MLPs on images

6 **Build** & train a simple CNN for flower classification

Today's Roadmap

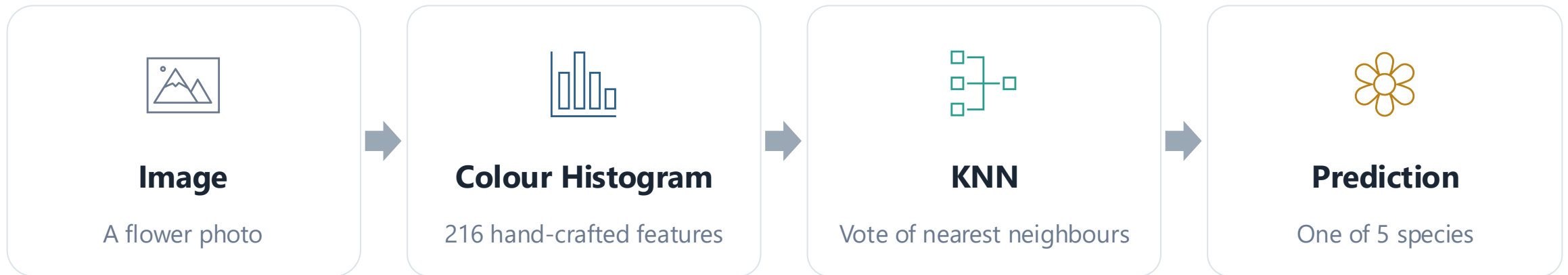
From last week's KNN to modern CNNs.



KEY IDEA One thread all day: from a classifier you **train** (MLP) to one that **learns its own features** (CNN).

Review: The Week 3 Pipeline

Last week: turn a flower photo into features, then let KNN vote on the answer.



THINK KNN works — but it is slow at prediction and can't learn better features. What if that isn't enough?

Limitations of KNN

KNN is a solid baseline — but its weaknesses are exactly what neural networks fix.

Strengths

- ✓ Simple to understand
- ✓ No training step needed

Limitations

- ✗ Slow at inference/prediction time
- ✗ Sensitive to feature quality
- ✗ Can't learn better features

THINK KNN never improves its features. Can a model **learn its own decision boundary**?

Task 1

1

Section 2 of 4

Task 1: MLP for Image Classification

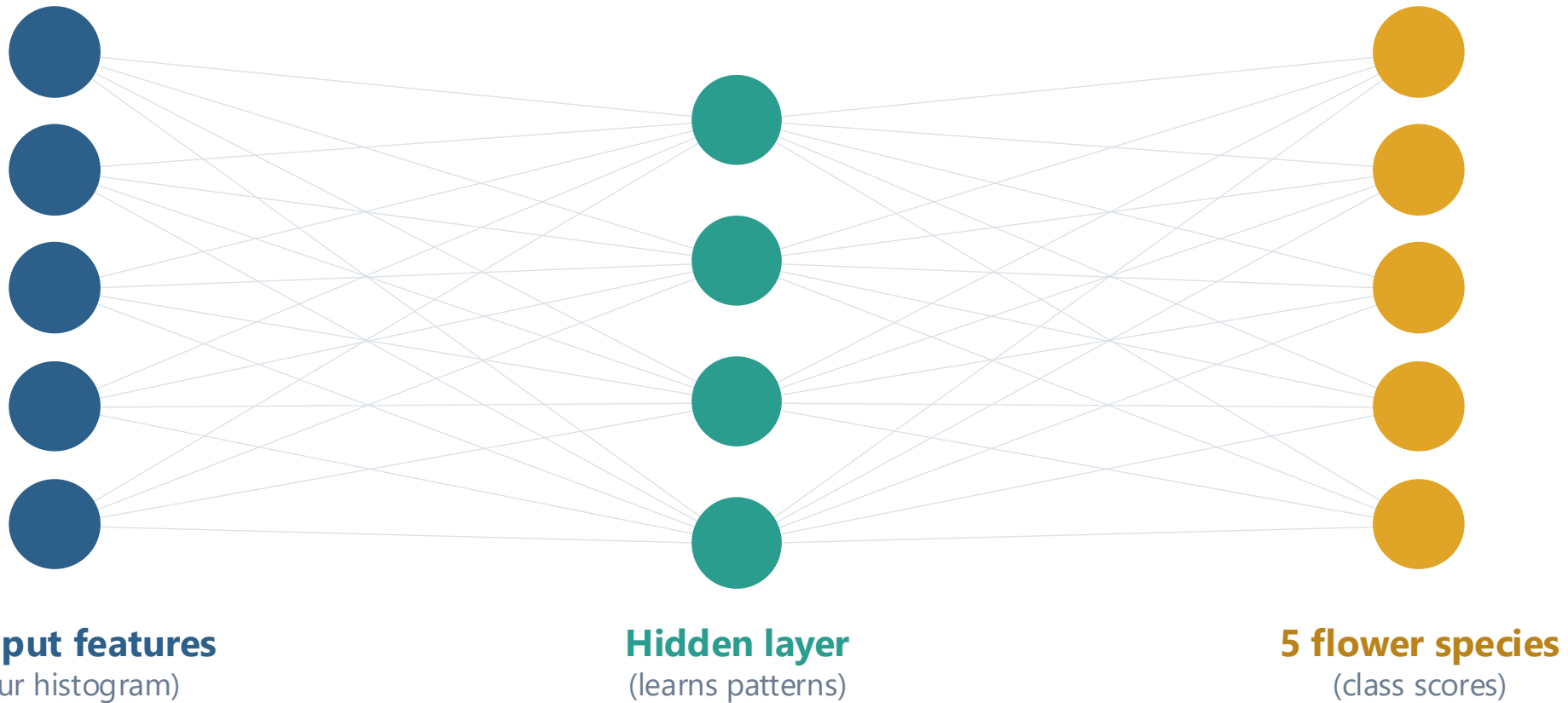
Classify images using a multi-layer perceptron (MLP).

IN THIS SECTION

MLP Architecture · Train NNs with Backprop & Gradient Descent

Introducing the Multi-Layer Perceptron

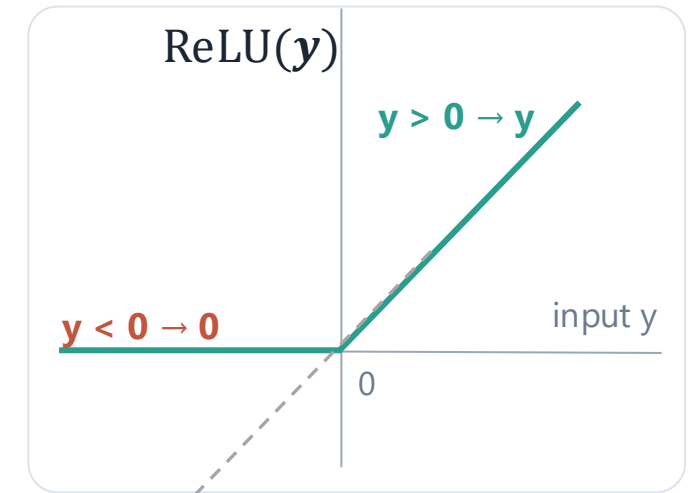
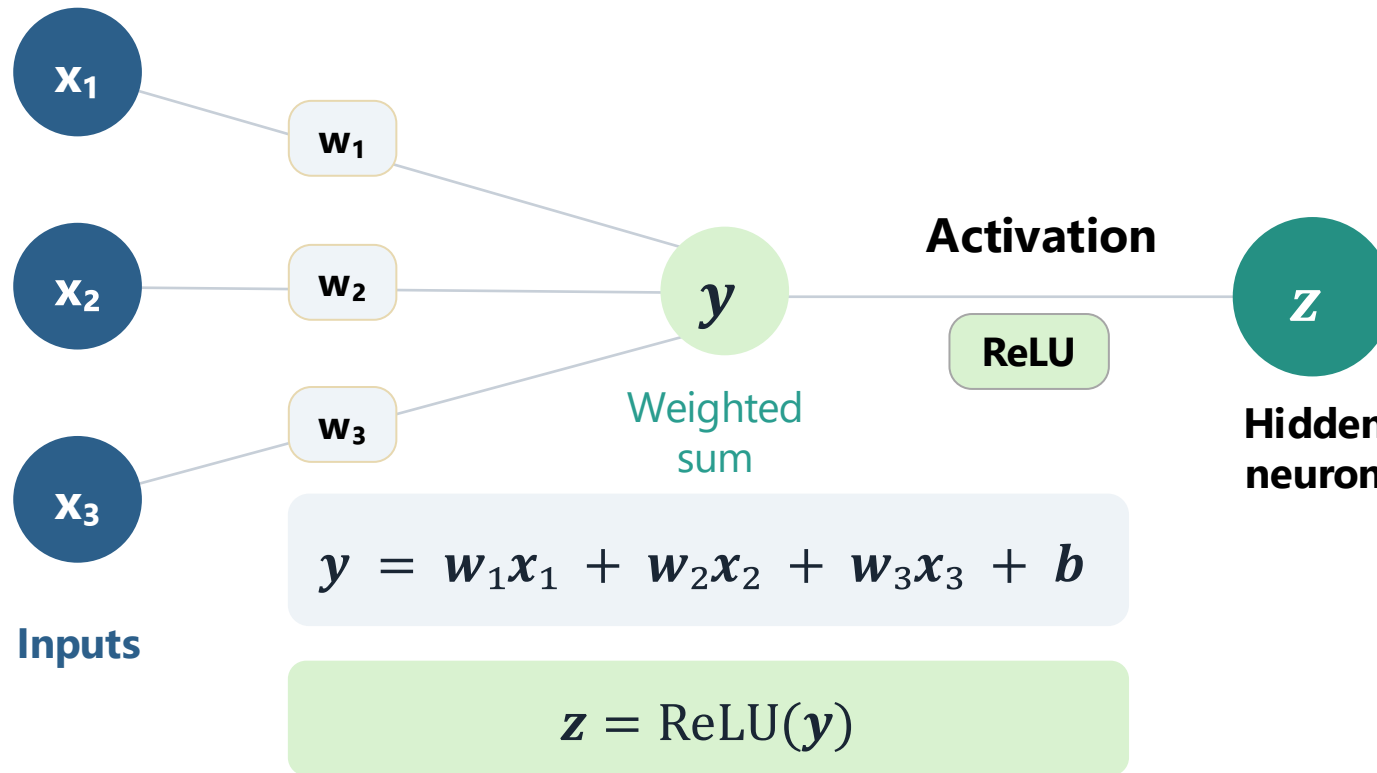
An MLP is just layers of neurons — features flow forward into a prediction.



KEY IDEA An MLP learns the mapping from features to class scores — no hand-written rules.

Inside an MLP — The Linear Layer

A neuron is just a placeholder to store a number

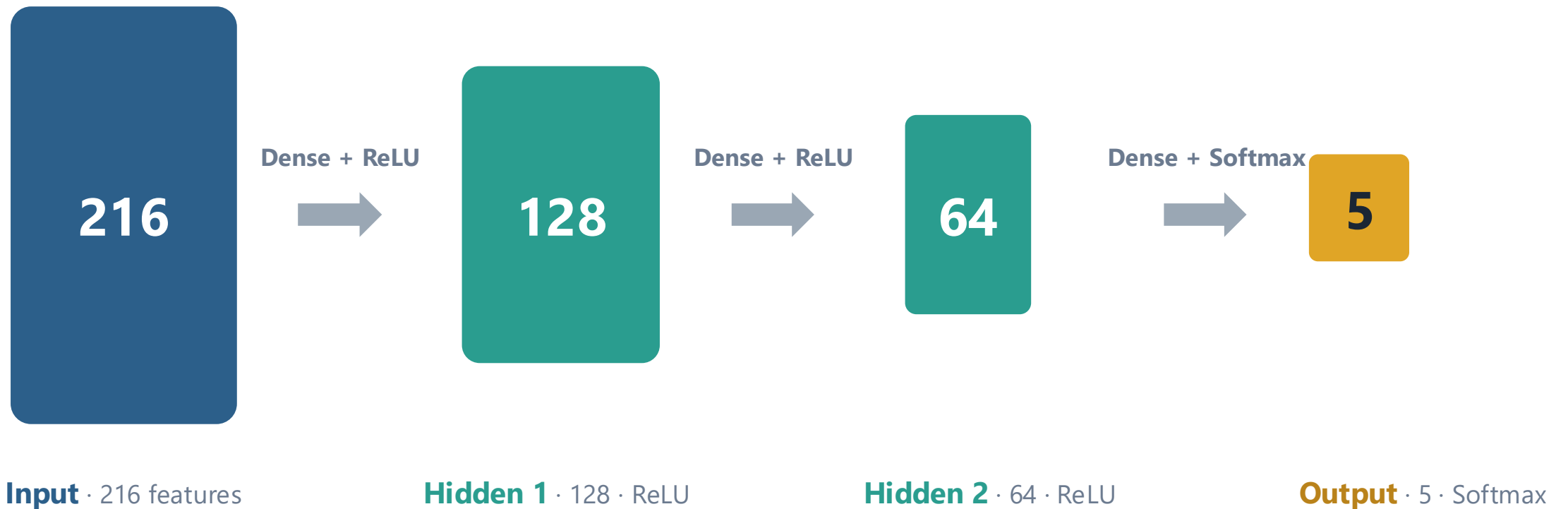


Activations introduce
non-linearity

KEY IDEA A neuron is just a weighted sum of its inputs, squashed by activation — the **weights** and **bias** are what the network learns.

Building an MLP

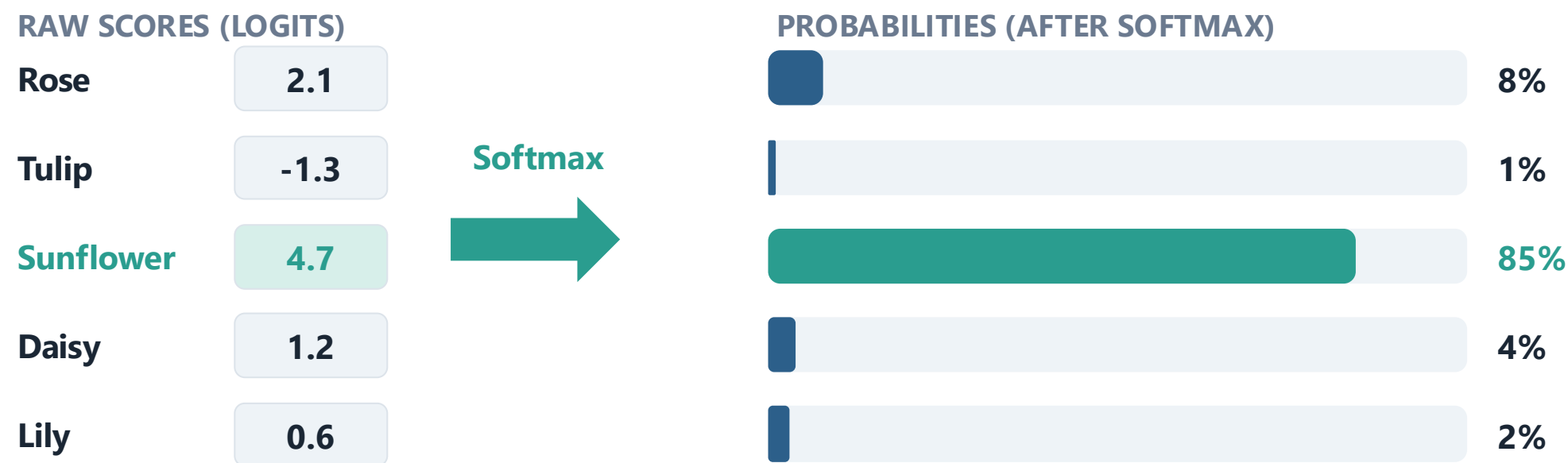
Stack layers — each one shrinks 216 features toward 5 class scores.



KEY IDEA Input → hidden layers with ReLU → output. Each layer learns a smaller, more useful representation.

Softmax: From Scores to Probabilities

Softmax converts the network's output scores into class probabilities.

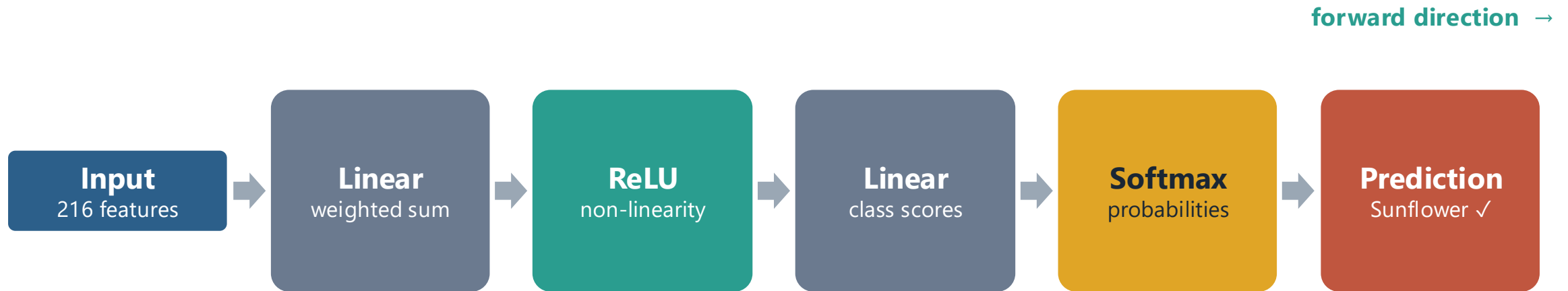


All five probabilities add up to 100%

KEY IDEA Softmax turns raw scores into probabilities that add to 100% — the highest one becomes the prediction.

The Forward Pass

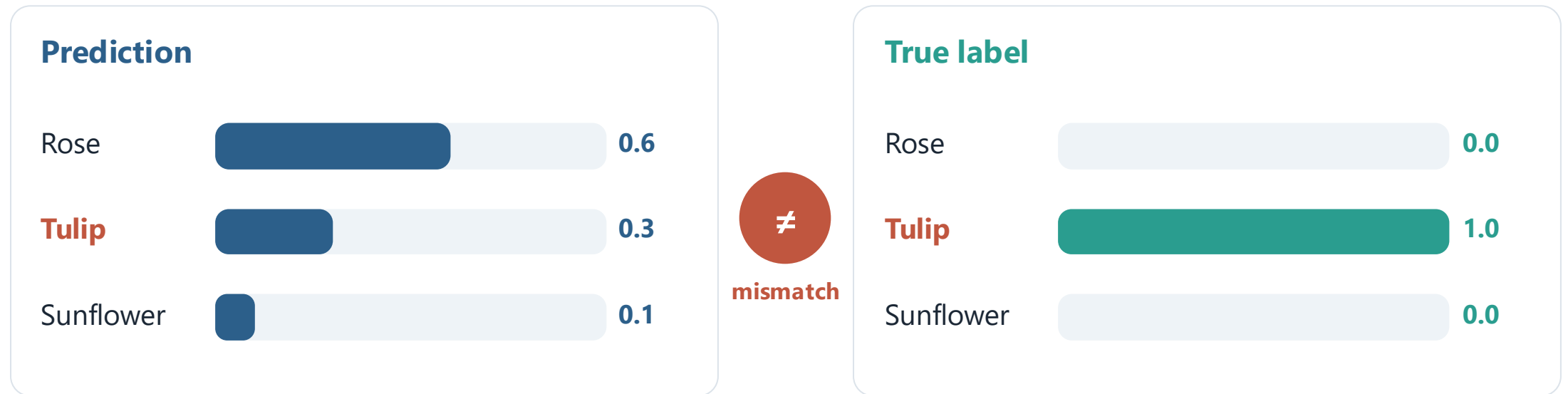
Run the input through every layer to produce a prediction.



KEY IDEA The forward pass just makes a guess. Learning happens next — by measuring how wrong that guess is.

The Loss Function

How wrong is the prediction? The loss puts a number on it.

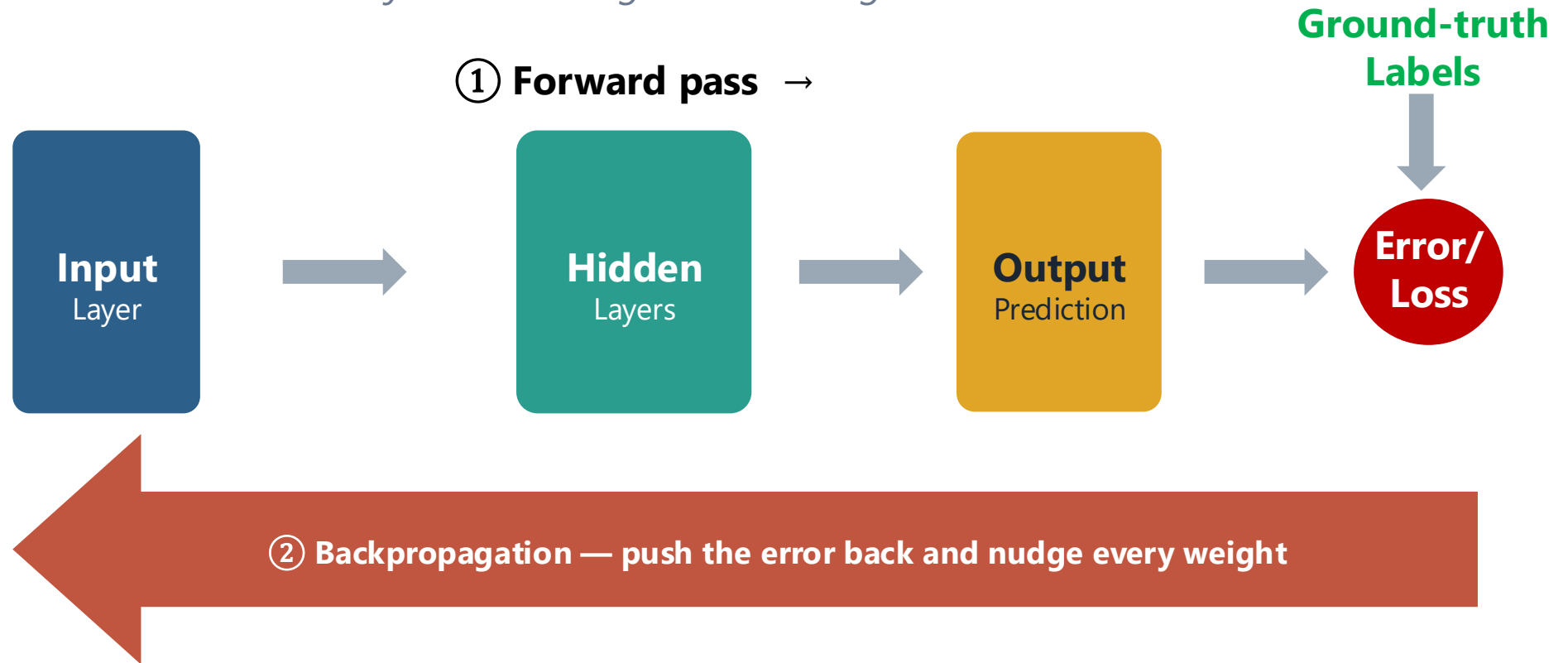


Cross-entropy loss is **high** here — the true class 'Tulip' only reached 0.3 instead of 1.0.

KEY IDEA The loss turns 'how wrong is this guess?' into one number the network can shrink/minimize.

Backpropagation

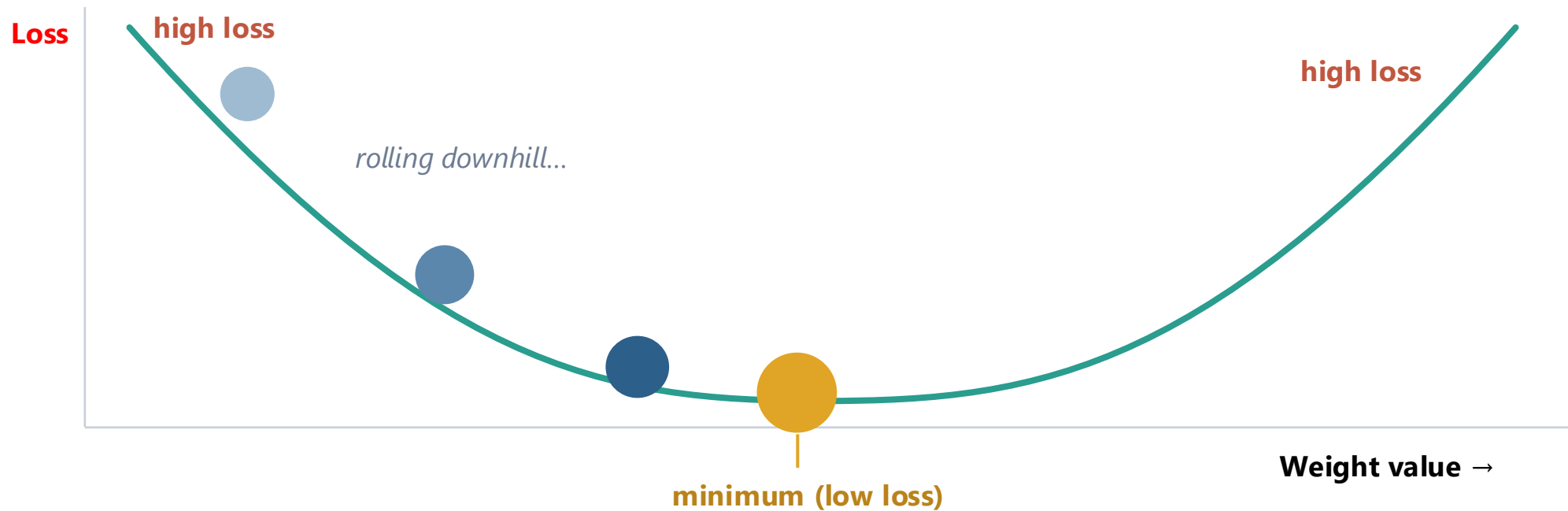
Use the error to work out how to adjust each weight — working backward.



KEY IDEA Backprop finds how each weight affected the error — no hand-tuning, just follow the error backward. Technically, it computes **gradient** of the loss w.r.t each parameter (weight, bias).

Gradient Descent

Picture the loss as a hill — training rolls the weights downhill toward the lowest point.

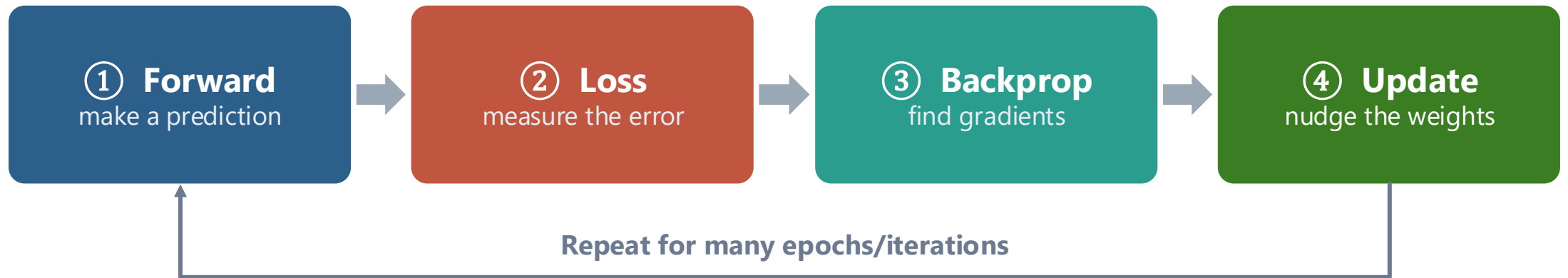


$$\text{new weight} = \text{old weight} - \text{learning rate} \times \text{slope (gradient)}$$

KEY IDEA Gradient descent nudges the weights downhill on the loss curve — every step makes the loss a little smaller.

The Training Loop

One pass of learning, repeated thousands of times — this single loop is deep learning.



KEY IDEA Forward → loss → backprop → update, over and over. That repeated loop is how a NN learns.

Implementing an MLP in Keras

The same network we designed — now in a few lines of Keras.

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(128, activation="relu", input_shape=(216,)),
    Dense(64, activation="relu"),
    Dense(5, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=20)
model.evaluate(X_test, y_test)
```

Dense() Fully-Connected / Linear layer

activation ReLU or Softmax

compile() pick loss + optimizer

fit() runs the training loop

evaluate() measures test accuracy

KEY IDEA Keras turns our diagram into code: define the layers, then **compile · fit · evaluate**.

TASK 2

2

Section 3 of 4

Task 2: CNN for Image Classification

Learn features directly from raw pixels with CNNs.

IN THIS SECTION

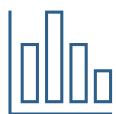
Convolutional & Pooling Layers • Build & Train a CNN

The MLP Workflow

Task 1 end to end: colour histogram $\rightarrow$ MLP $\rightarrow$ prediction.

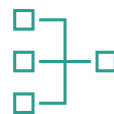


Still hand-crafted



Colour Histogram

216 hand-crafted numbers



MLP

learns the classifier



Prediction

Sunflower ✓

THINK The MLP learns the decision boundary — but we still *design the features by hand*.
Could the network learn the features too?

The Problem with Colour Histograms

Colour histograms throw away **where** things are — so different shapes can look identical.

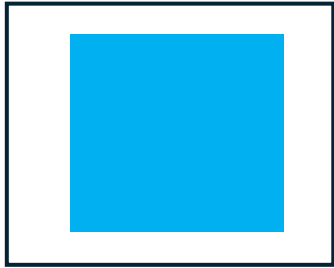
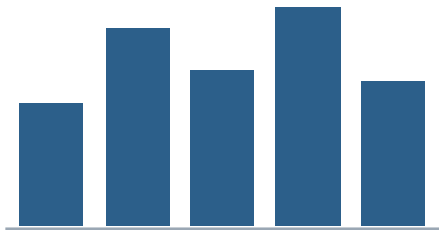


Image A · square



colour histogram



different shape

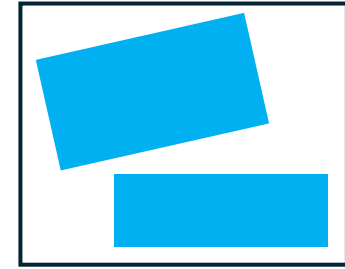
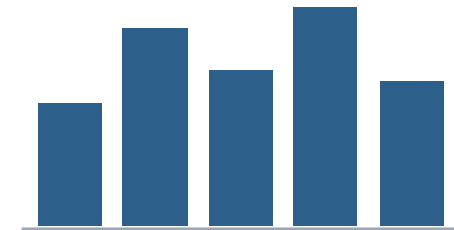
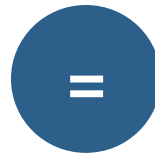


Image B · **two rectangles**



colour histogram

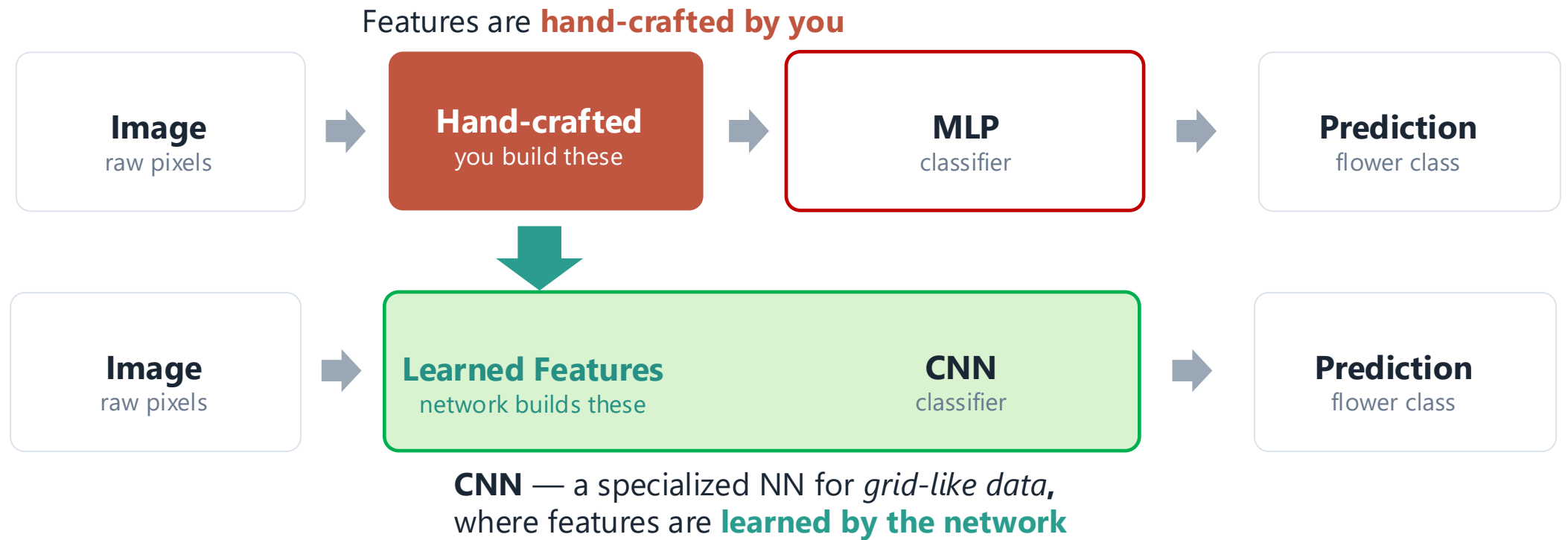


same histogram

THINK Same colours → same histogram. To an MLP, these two images are identical — yet their shapes differ.
How could a model use shape?

The Idea of a CNN

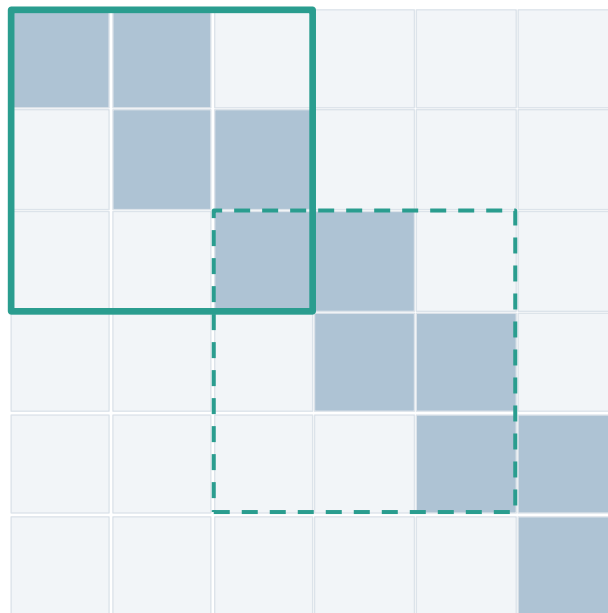
The big leap: stop hand-crafting features — let the network discover them from pixels.



KEY IDEA CNNs replace hand-crafted features with features learned straight from the pixels.

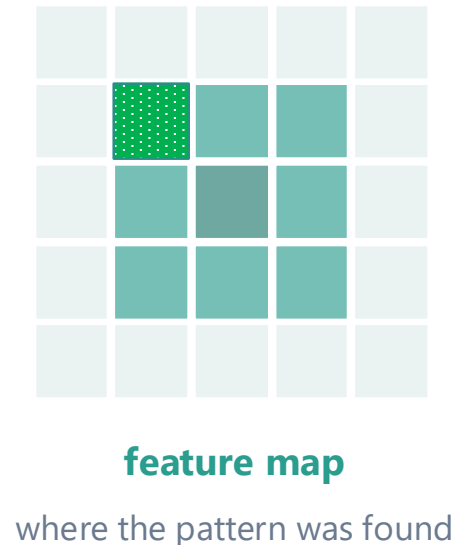
Convolution: Filters That Scan the Image

Convolution scans the image with small filters that respond to local patterns.



3×3 filter slides across the image

convolve
→
Activation
(ReLU)



A filter can detect:

Edges

sharp boundaries

Corners

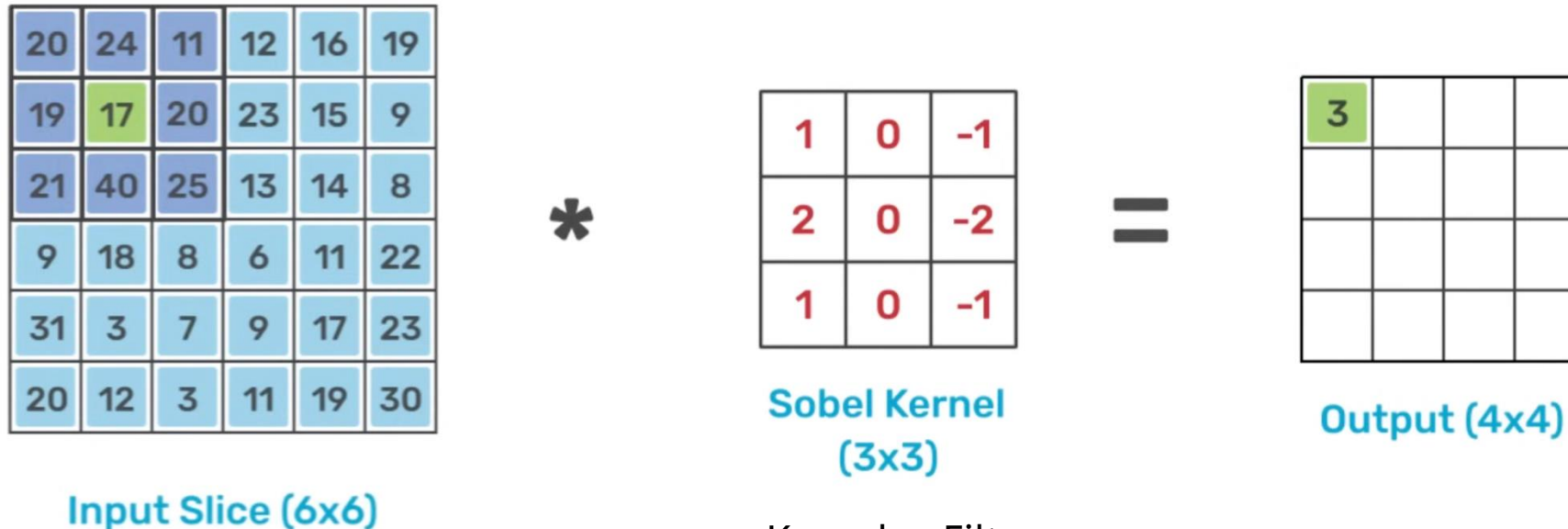
where edges meet

Textures

repeated patterns

KEY IDEA A convolution slides a small filter over the image to build a feature map — the network **learns** what each filter detects.

Conv2D: Explanation & Example



Kernel = Filter,
where **weights** is learned

Source: <https://www.youtube.com/watch?v=yb2tPt0QVPY>

2D Convolution Explained: Fundamental Operation in Computer Vision

Max Pooling

Pooling summarises each region — keeping the strongest signal and shrinking the map.

1	3	2	4
5	6	1	2
7	2	8	9
1	0	3	4

feature map · 4×4

max-pool 2×2



6	4
7	9

pooled · 2×2

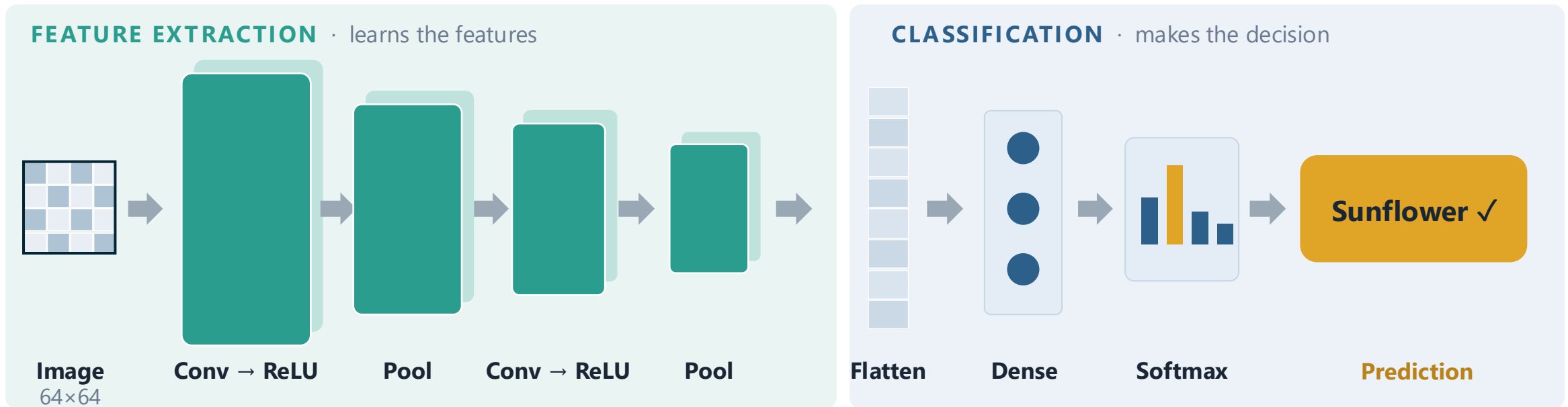
Strongest
keeps the peak value

Smaller
½ width & height

KEY IDEA Max pooling keeps the strongest response in each patch — shrinking the map while keeping what matters.

CNN Architecture

A CNN stacks convolution + pooling to learn features, then dense layers to classify.



KEY IDEA Convolution + pooling learn the features; the dense layers classify them — all trained together, end to end.

Implementing a CNN in Keras

The CNN we just designed — a few lines of Keras.

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
    Flatten, Dense
model = Sequential([
    Conv2D(32, (3,3), activation="relu",
        input_shape=(64,64,3)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(5, activation="softmax"),
])
model.compile(optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"])
model.fit(X_train, y_train, epochs=20)
```

Conv2D() learns filters from pixels

MaxPooling2D() down-samples the maps

Flatten() grid → 1-D vector

Dense() classifies the features

compile · fit train end-to-end

KEY IDEA Conv2D + pooling learn the features; Dense layers classify — all trained with **compile · fit**.

Why CNNs Beat MLPs on Images

Same task, very different design — here is why CNNs win on images.

MLP

- ✗ Needs hand-crafted features
- ✗ Ignores spatial structure
- ✗ Many parameters to train
- ✗ Weak on images

CNN

- ✓ **Learns features from pixels**
- ✓ **Uses spatial structure**
- ✓ **Parameter sharing — efficient**
- ✓ **Excellent on images**

KEY IDEA CNNs are built for images — they learn features, respect spatial structure, and share parameters.

WRAP-UP



Section 4 of 4

Review & Reflection



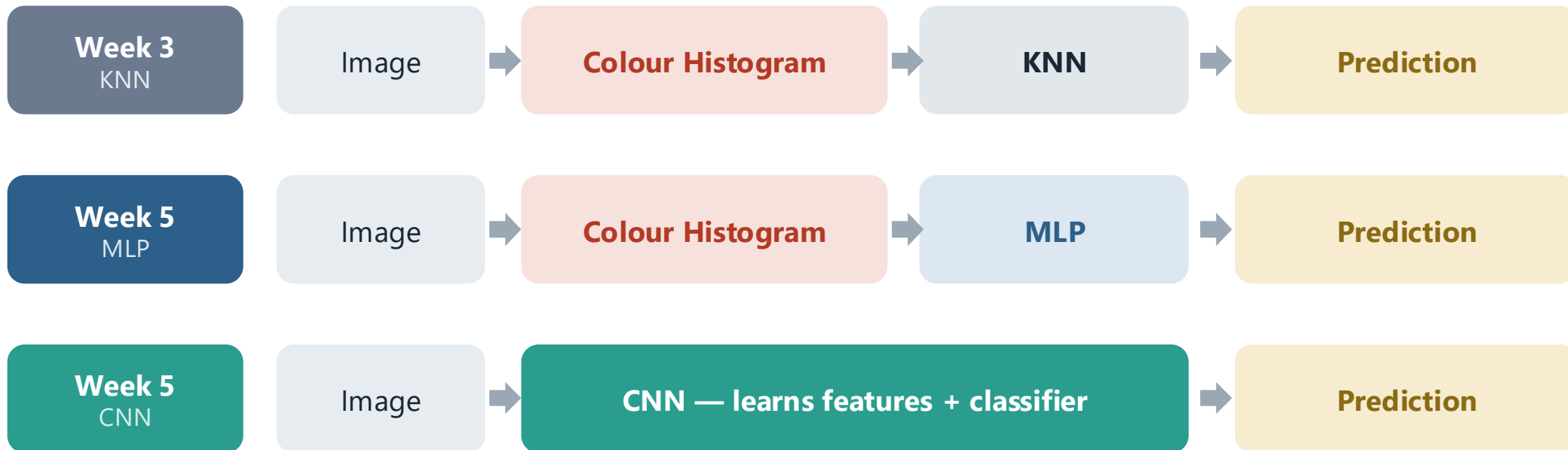
MLP vs CNNs takeaways.

IN THIS SECTION

What we learned · Reflection questions

The Evolution: KNN → MLP → CNN

Three ways to classify the same flower — and how far we have come.



KEY IDEA Hand-crafted features gave way to learned ones — you can now explain **and** build both MLPs and CNNs.

Reflection and Questions

Consolidate what you built across Task 1 (MLP) and Task 2 (CNNs).

1. How did moving from an MLP to a CNN change the way features are learned from raw pixels?
2. Which hyperparameters (learning rate, hidden layers, filters) had the biggest impact on accuracy — and why?
3. Where did your model overfit, and which techniques helped you reduce it?
4. How do backpropagation and gradient descent actually update the network's weights during training?
5. If you had more time or data, what single change would most improve your classifier?